Polymorphism :

Polymorphism means having multiple forms of one thing. In inheritance, polymorphism is done, by function overriding, when both parent and child class have member function with same name , declaration but different definition.

Function Overriding :

Function overriding is the mechanism using which a function defined in the base class is once again defined in the derived class. In this case, we say the function is overridden in the derived class.

Requirements for Overriding a Function

- Inheritance should be there. Function overriding cannot be done within a class. For this we require a derived class and a base class.
- Function that is redefined must have exactly the same declaration in both base and derived class, that means same name, same return type and same type of input arguments .

Example of Function Overriding in C++

```
class Base
{
    public:
    void show()
    {
        cout << "Base class";
    }
};

class Derived:public Base
{
    public:
    void show()  // function overriding from Base class
    {
        cout << "Derived Class";
    }
};
```

Vector India

```
main()

{    Base bobj;

     Derived dobj;

     bobj.show();  // Early Binding

     dobj.show(); // Early Binding

}

output :

     Base class

     Derived class
```

In the above program, we have a base class and a derived class. In the base class, we have a function show which is overridden in the derived class. In the main function, we create an object each of Base and Derived class and call the show function with each object. It produces the desired output.

The above binding of functions using objects of each class is an example of Early binding.

Matching the function call with the function definition is called ***Binding.***

If Binding is done at compile time then it is called **Early Binding** .Early Binding is also know as Static Binding or Compile time Binding.

Examples of Early Binding :

1. Function Over Loading

2. Operator Over Loading.

If Binding is done at run time then it is called **Late Binding** .Late Binding is also know as Dynamic Binding or Run time Binding.

Example :

1.Function Overriding

Vector India

Now let us see what happens when we use the base class pointer and assign derived class objects as its contents.

The example program is shown below:

```
class Base
{
    public:
    void show()
    {
        cout << "Base class";
    }
};

class Derived:public Base
{
    public:
    void show()  // function overriding from Base class
    {
        cout << "Derived Class";
    }
};

main()

{

    Base *ptr=NULL;  // Base class pointer

    Derived dobj;    // Derived class object

    ptr=&dobj;     //  Base class pointer holding the address of
                   derived class object

     ptr->show(); // Early Binding.

}
```

Output :

Class::Base

Vector India

Now we see, that the output is “Class:: Base”. So irrespective of what type object the base pointer is holding, the program outputs the contents of the function of the class whose base pointer is the type of. In this case, also static linking is carried out.

In order to make the base pointer output, correct contents and proper linking, we go for dynamic binding of functions. This is achieved using Virtual functions.

Virtual Function is a function in base class, which is overrided in the derived class, and which tells the compiler to perform **Late Binding** on this function.

`Virtual` Keyword is used to make a member function of the base class Virtual.

Using Virtual Keyword

We can make base class's methods virtual by using **virtual** keyword while declaring them. Virtual keyword will lead to Late Binding of that method.

```
class Base
{
    public:
    virtual void show()
    {
        cout << "Base class\n";
    }
};

class Derived:public Base
{
    public:
    void show()
    {
        cout << "Derived Class";
    }
}

int main()
{
    Base* ptr=NULL;       //Base class pointer
    Derived obj;     //Derived class object
    ptr=&obj;
    ptr->show();     //Late Binding Occurs
}
```

Output :

Vector India

Derived class

On using Virtual keyword with Base class's function, Late Binding takes place and the derived version of function will be called, because base class pointer pointes to Derived class object.

Every class that uses virtual functions (or is derived from a class that uses virtual functions) is given its own virtual table.

This table is simply a static array that the compiler sets up at compile time.

A virtual table contains one entry for each virtual function that can be called by objects of the class. Each entry in this table is simply a function pointer that points to the most-derived function accessible by that class.

Second, the compiler also adds a hidden pointer to the base class, which we will call *__vptr or virtual pointer .

Virtual pointer is set (automatically) when a class instance is created so that it points to the virtual table for that class.

Unlike the *this pointer, which is actually a function parameter used by the compiler to resolve self-references, virtual pointer is a real pointer. Consequently, it makes each class object allocated bigger by the size of one pointer. It also means that virtual pointer is inherited by derived classes, which is important.

Example program:

```
class Base
{
    public :
        virtual void function1()
        {
            cout<<”Base Function1:”<<endl;
        }
        virtual void function2()
        {
            cout<<”Base Function2:”<<endl;
        }
```

Vector India

```
};
class D1 :public Base
{
	public : void function1()
		{
			cout<<"D1 function1:"<<endl;
		}
};

class D2 :public Base
{
	public : void function2()
		{
			cout<<"D2 function2:"<<endl;
		}
};
```

Because there are 3 classes here, the compiler will set up 3 virtual tables: one for Base, one for D1, and one for D2.

The compiler also adds a hidden pointer to the most base class that uses virtual functions. Although the compiler does this automatically, we'll put it in the next example just to show where it's added:

```
class Base
{
	public :
		FunctionPointer *__vptr; // hidden pointer (Virtual pointer)added by compiler
		virtual void function1()
		{
```


Vector India

```
                cout<<"Base Function1:"<<endl;
        }
        virtual void function2()
        {
                cout<<"Base Function2:"<<endl;
        }
};
class D1 :public Base
{
    public : void function1()
        {
                cout<<"D1 function1:"<<endl;
        }
};

class D2 :public Base
{
    public : void function2()
        {
                cout<<"D2 function2:"<<endl;
        }
};
```

When a class object is created, *__vptr is set to point to the virtual table for that class. For example, when a object of type Base is created, *__vptr is set to point to the virtual table for Base. When objects of type D1 or D2 are constructed, *__vptr is set to point to the virtual table for D1 or D2 respectively.

Now, let's talk about how these virtual tables are filled out. Because there are only two virtual functions here, each virtual table will have two entries (one for function1(), and one for function2()). Remember that when these

Vector India

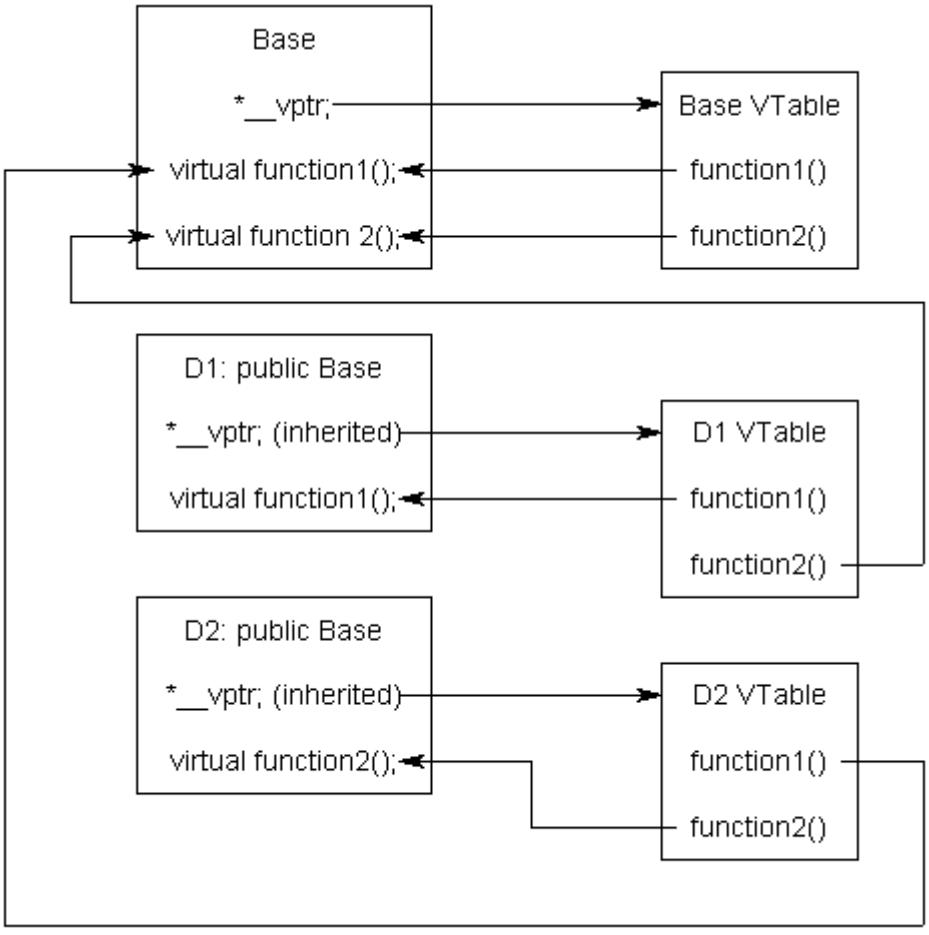
virtual tables are filled out, each entry is filled out with the most-derived function an object of that class type can call.

The virtual table for Base objects is simple. An object of type Base can only access the members of Base. Base has no access to D1 or D2 functions. Consequently, the entry for function1 points to Base::function1(), and the entry for function2 points to Base::function2().

The virtual table for D1 is slightly more complex. An object of type D1 can access members of both D1 and Base. However, D1 has overridden function1(), making D1::function1() more derived than Base::function1(). Consequently, the entry for function1 points to D1::function1(). D1 hasn't overridden function2(), so the entry for function2 will point to Base::function2().

The virtual table for D2 is similar to D1, except the entry for function1 points to Base::function1(), and the entry for function2 points to D2::function2().

Here's a picture of this graphically:



the virtual ptr in each class points to the virtual table for that class. The entries in the virtual table point to the most-derived version of the function objects of that class are allowed to call.

So consider what happens when we create an object of type D1:

```
int main()
{
	D1  d1;
}
```

Because d1 is a D1 object, d1 has its *__vptr set to the D1 virtual table.

Now, let's set a base pointer to D1:

```
int main()
{
	D1  d1;
	Base  *dPtr=&d1;
	return 0;
}
```

Note that because dPtr is a base pointer, it only points to the Base portion of d1. However, also note that *__vptr is in the Base portion of the class, so dPtr has access to this pointer. Finally, note that dPtr->__vptr points to the D1 virtual table! Consequently, even though dPtr is of type Base, it still has access to D1's virtual table (through __vptr).

So what happens when we try to call dPtr->function1()?

```
int main()
{
	D1  d1;
	Base  *dPtr=&d1;
```

Vector India

```
    dPtr->function1();
    return 0;
}
```

First, the program recognizes that function1() is a virtual function. Second, the program uses dPtr->__vptr to get to D1's virtual table. Third, it looks up which version of function1() to call in D1's virtual table. This has been set to D1::function1(). Therefore, dPtr->function1() resolves to D1::function1()!

Now, you might be saying, "But what if dPtr really pointed to a Base object instead of a D1 object. Would it still call D1::function1()?". The answer is no.

```
int main()
{
    Base  b;
    Base  *dPtr=&b;
    dPtr->function1();
    return 0;
}
```

In this case, when b is created, __vptr points to Base's virtual table, not D1's virtual table. Consequently, bPtr->__vptr will also be pointing to Base's virtual table. Base's virtual table entry for function1() points to Base::function1().

Thus, bPtr->function1() resolves to Base::function1(), which is the most-derived version of function1() that a Base object should be able to call.

By using these tables, the compiler and program are able to ensure function calls resolve to the appropriate virtual function, even if you're only using a pointer or reference to a base class!

Calling a virtual function is slower than calling a non-virtual function for a couple of reasons: First, we have to use the *__vptr to get to the appropriate virtual table. Second, we have to index the virtual table to find the correct function to call. Only then can we call the function.

Also as a reminder, any class that uses virtual functions has a __vptr, and thus each object of that class will be bigger by one pointer. Virtual functions are powerful, but they do have a performance cost.

Vector India